2026-7-17 Linux文件IO--文件进程字节流

- 1. 从磁盘对象到进程视角：理解文件
  - 1.1 文件的狭义理解
    - 1.1.1 文件首先是磁盘上的持久化对象
    - 1.1.2 文件读写最终属于 I/O
  - 1.2 文件的广义理解
    - 1.2.1 Linux 以统一的文件抽象描述资源
    - 1.2.2 文件抽象连接了用户与操作系统
  - 1.3 文件由内容和属性共同构成
    - 1.3.1 空文件也拥有文件属性
    - 1.3.2 所有文件操作都可以归为内容和属性两类
  - 1.4 文件必须进入进程视角
    - 1.4.1 文件由进程打开
    - 1.4.2 访问文件前必须先定位文件
    - 1.4.3 打开文件后，操作系统需要管理内存中的文件对象
  - 1.5 从 /proc 观察进程与文件的关系
    - 1.5.1 查看进程链接的当前目录和可执行程序
    - 1.5.2 可执行文件、进程和打开文件是三件事
- 2. C 标准库文件操作
  - 2.1 FILE 与文件操作闭环
    - 2.1.1 C 语言使用 FILE 表示文件流
    - 2.1.2 文件模式决定“能否读写、是否创建、是否清空”
  - 2.2 用 w 清空文件
    - 2.2.1 fopen(..., "w") 自带截断语义
    - 2.2.2 输出重定向与 w 模式具有相同的结果
  - 2.3 写入与追加
    - 2.3.1 使用 fputs 或 fwrite 写内容
    - 2.3.2 r+ 与 a+ 的区别
  - 2.4 读取文件内容
    - 2.4.1 使用 fgets 按行读取
    - 2.4.2 使用 fread 读取任意字节
    - 2.4.3 文件内容可以看作字节序列
  - 2.5 向显示器输出信息
    - 2.5.1 C 标准库提供多种输出方式
    - 2.5.2 标准输入、标准输出和标准错误
  - 2.6 正确关闭文件

        - 2.6.1 fclose 结束一次文件流的使用
- 3. 文件位置与基础读写接口回顾
    - 3.1 读写位置的直观理解
        - 3.1.1 文件是一维字节空间
        - 3.1.2 追加模式固定从末尾写入
    - 3.2 C 标准库的文件位置接口
        - 3.2.1 ftell 获取当前位置
        - 3.2.2 fseek 设置当前位置
        - 3.2.3 rewind 回到文件开头
    - 3.3 本章边界

1. 从磁盘对象到进程视角：理解文件

1.1 文件的狭义理解

1.1.1 文件首先是磁盘上的持久化对象

从最直观的层面看，文件存放在磁盘中。磁盘是外部存储设备，断电后数据仍然可以保留，因此磁盘上的文件具有持久性。

磁盘本身是外设，所以对磁盘文件的读写，最终都属于对“外部设备”的输入和输出，简称 I/O。

1.1.2 文件读写最终属于 I/O

用户程序通常会调用 C/C++ 等语言提供的文件接口，但这些库函数并不是文件操作的最底层。它们的价值在于把不同操作系统中的文件接口包装成相对统一的用法。

因此，理解文件不能只停留在 `fopen`、`fread`、`fwrite` 这些函数名上，还需要继续追问：

1. 文件究竟是磁盘上的什么对象？
2. 谁来访问它？
3. 访问前为什么要先打开？
4. 语言层的一次读写，最终是谁在操作磁盘？

1.2 文件的广义理解

1.2.1 Linux 以统一的文件抽象描述资源

在 Linux 中，文件不仅是磁盘上的普通数据和源代码文件，也是一套统一的资源访问抽象。

键盘、显示器、网卡、磁盘、管道等对象，都可以通过类似文件的方式被描述和访问。用户不必为每一种设备重新学习一套完全不同的接口。

> **Note**
>
> “一切皆文件”强调的是一种统一抽象，并不意味着所有对象在物理上都是磁盘文件。

1.2.2 文件抽象连接了用户与操作系统

普通文件、字符设备和块设备等对象的访问方式可能不同，但用户程序面对的接口可以保持一致。这样，文件就成为用户进程访问大量系统资源的一种统一入口。

1.3 文件由内容和属性共同构成

1.3.1 空文件也拥有文件属性

即使新建一个大小为 0 字节的空文件，它在磁盘上仍然是一个真实存在的对象。文件内容虽然为空，但文件名、位置、权限、时间等属性仍然需要被记录。

所以：

> **文件 = 文件内容 + 文件属性（元数据）**

1.3.2 所有文件操作都可以归为内容和属性两类

围绕上面的等式，文件操作也可以分为两类：

1. 修改文件内容，例如读写文本、复制数据；
2. 修改文件属性，例如重命名、修改权限、更新时间。

“空文件是否占用磁盘空间”这个问题，本质上是在区分“文件对象及其元数据是否存在”和“文件的数据内容是否为空”。空文件的内容为 0 字节，但文件对象本身仍然需要被文件系统记录。

1.4 文件必须进入进程视角

1.4.1 文件由进程打开

一段代码被编译成可执行文件，并不会自动打开其他文件。只有程序运行起来形成进程，并执行到打开文件的语句时，文件才真正进入“被进程使用”的状态。

因此，课堂中讨论的文件操作，本质上是在讨论：

> **进程和文件之间的关系。**

1.4.2 访问文件前必须先定位文件

要访问一个文件，进程首先需要知道它在哪里。路径就是定位文件的一种方式。

编写下面这段代码时，文件名没有带目录：

```
FILE *fp = fopen("myfile", "w");
```

系统仍然知道文件应该创建到哪里，因为进程启动后会维护一个**当前工作目录**（current working directory，简称 `cwd`）。

上面的相对路径会被解释为：

```
<当前工作目录>/myfile
```

所以，不写路径并不表示没有路径。程序要么由用户提供路径，要么使用进程的 `cwd` 补全路径。改变当前工作目录，同名文件对应的实际位置也会随之改变。

1.4.3 打开文件后，操作系统需要管理内存中的文件对象

Linux 中同时可能存在大量被打开的文件。可以把文件按位置粗略分为：

1. 内存中正在被进程使用的打开文件；
2. 磁盘上尚未被打开的文件。

进程要访问文件时，操作系统需要把相关元数据和所需的数据页纳入内存中的管理结构。此处不能简单理解成“每次打开都把整份文件一次性搬入进程内存”，但对文件内容的访问确实要经过内存中的缓存和管理结构。

操作系统管理这些打开文件时，遵循“先描述，再组织”的思路：

1. **描述**：用数据结构记录文件属性、读写位置、访问状态等信息；
2. **组织**：把这些描述对象放入链表、索引表等结构中，从而支持查找、新增和删除。

具体的数据结构、文件描述符与进程打开文件表将在下一章展开。

1.5 从 `/proc` 观察进程与文件的关系

1.5.1 查看进程链接的当前目录和可执行程序

Linux 的 `/proc` 文件系统可以观察运行中进程的状态。假设当前进程的 PID 为 `<pid>`，可以执行：

```
ls -l /proc/<pid>/
ls -l /proc/<pid>/cwd
ls -l /proc/<pid>/exe
```

其中：

- `cwd` 是指向进程当前工作目录的符号链接；
- `exe` 是指向启动该进程的可执行文件完整路径的符号链接。

例如，程序使用相对路径 `"myfile"` 打开文件时，系统会结合 `cwd` 得到真实的访问位置。这正好说明了“文件名不带路径”时，系统仍然能够找到文件的原因。

1.5.2 可执行文件、进程和打开文件是三件事

三者可以这样区分：

1. 可执行文件存放在磁盘上；
2. 运行后形成进程，进程拥有自己的状态和当前工作目录；
3. 进程执行打开操作后，文件才成为该进程正在使用的打开文件。

> 板书给出了从“理解文件”到系统 I/O、文件描述符、重定向和缓冲区的完整路线；本章只整理其中的文件概念与 C 文件接口。

2. C 标准库文件操作

2.1 `FILE *` 与文件操作闭环

2.1.1 C 语言使用 `FILE *` 表示文件流

C 标准库通过 `FILE *` 表示一个文件流。常见流程是：

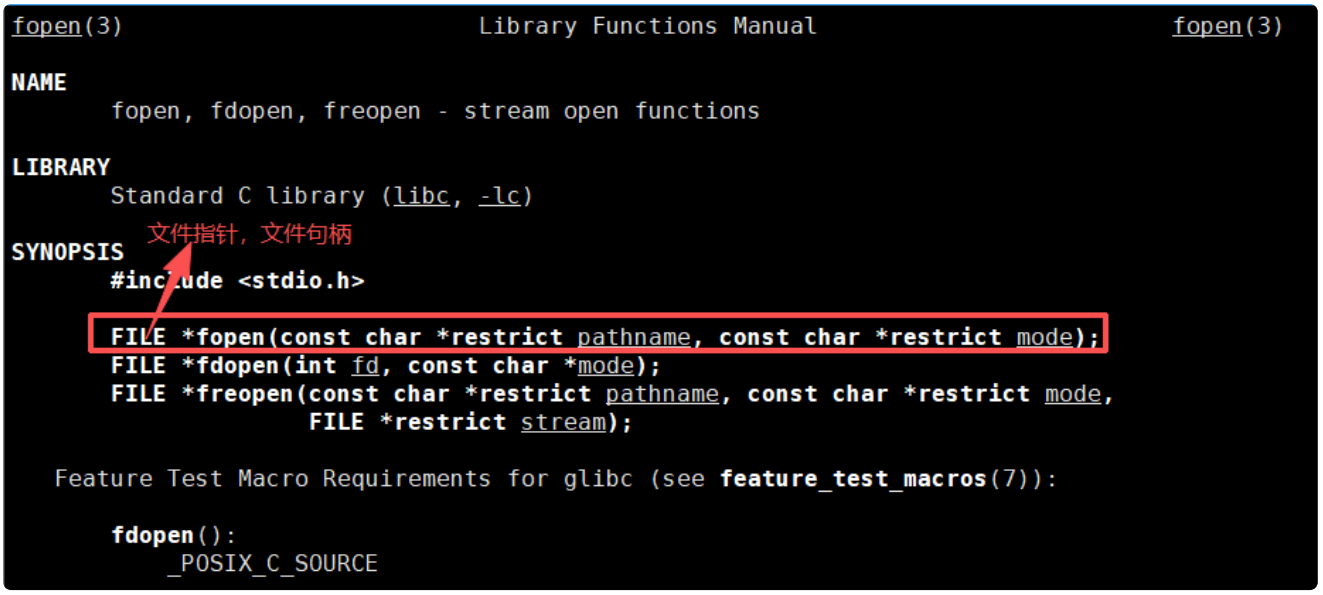
1. 使用 `fopen` 打开文件；
2. 检查 `fopen` 是否返回 `NULL`；
3. 使用读写接口处理文件；
4. 使用 `fclose` 关闭文件。

```c
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("myfile", "w");
    if (fp == NULL)
    {
        perror("fopen");
        return 1;
    }

    fputs("hello bit!\n", fp);
    fclose(fp);
    return 0;
}
```

`fopen` 失败时会返回空指针。实际代码应当检查返回值，否则后续对空指针进行操作可能造成程序异常。



2.1.2 文件模式决定“能否读写、是否创建、是否清空”

使用 `fopen` 时，第二个参数决定文件的打开模式。除了能否读写之外，还要特别注意两件事：

1. 文件不存在时，模式是否允许创建；
2. 文件已经存在时，新模式是保留、清空，还是从末尾追加。

模式	读取	写入	文件不存在时	文件已存在时	初始位置
`r`	可读	不可写	打开失败	保留内容	文件开头
`r+`	可读	可写	打开失败	保留内容	文件开头
`w`	不可读	可写	创建文件	**清空内容**	文件开头
`w+`	可读	可写	创建文件	**清空内容**	文件开头
`a`	不可读	可追加	创建文件	保留并追加	文件末尾
`a+`	可读	可追加	创建文件	保留并追加	读取位置在开头，输出追加到末尾

```
r      Open text file for reading.  The stream is positioned at  the  beginning  of
       the file.

r+     Open  for reading and writing.  The stream is positioned at the beginning of
       the file.

w      Truncate file to zero length or create text file for writing.  The stream is
       positioned at the beginning of the file.

w+     Open for reading and writing.  The file is created if  it  does  not  exist,
       otherwise it is truncated.  The stream is positioned at the beginning of the
       file.

a      Open for appending (writing at end of file).  The file is created if it does
       not exist.  The stream is positioned at the end of the file.

a+     Open  for  reading and appending (writing at end of file).  The file is cre-
       ated if it does not exist.  Output is always appended  to  the  end  of  the
       file.   POSIX is silent on what the initial read position is when using this
       mode.  For glibc, the initial file position for reading is at the  beginning
       of  the file, but for Android/BSD/MacOS, the initial file position for read-
       ing is at the end of the file.
```

这里最容易误判的是 `w` 和 `a`：

- `w` 不只是“写”，还会在文件已存在时截断文件；
- `a` 不只是“写”，它会让输出固定在文件末尾，适合追加日志。

2.2 用 `w` 清空文件

2.2.1 `fopen(..., "w")` 自带截断语义

下面的程序没有写入任何内容，只以 `"w"` 模式打开文件再关闭：

```c
#include <stdio.h>

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Usage: %s filename\n", argv[0]);
        return 1;
    }

    FILE *fp = fopen(argv[1], "w");
    if (fp == NULL)
    {
        perror("fopen");
        return 1;
    }
```

```
    fclose(fp);
    return 0;
}
```

如果文件原本有内容，那么 `fopen(..., "w")` 会先把文件截断为 0 字节。关闭文件时虽然没有写入数据，原内容仍然已经消失。

```
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ make
gcc -o openfile openfile.c
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ ll
total 56
drwxrwxr-x  2 ZL ZL  4096 Jul 17 17:27 ./
drwxrwxr-x 17 ZL ZL  4096 Jul 17 14:56 ../
-rw-rw-r--  1 ZL ZL     4 Jul 17 17:21 aaaaa
-rwxrwxr-x  1 ZL ZL 16048 Jul 17 17:25 clearfile*
-rw-rw-r--  1 ZL ZL   110 Jul 17 16:35 log.txt
-rw-rw-r--  1 ZL ZL    70 Jul 17 14:58 Makefile
-rwxrwxr-x  1 ZL ZL 16048 Jul 17 17:27 openfile*
-rw-rw-r--  1 ZL ZL   519 Jul 17 17:27 openfile.c
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ rm clearfile
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ mv openfile clearfile
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ cat log.txt
hello word
hello word
hello word
hello word
hello word
hello word
hello word
hello word
hello word
hello word
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ ./clearfile
Usage: ./clearfile filename
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ ./clearfile log.txt
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ cat log.txt
ZL@VM-0-14-ubuntu:~/code/lesson13-file$
```

2.2.2 输出重定向与 `w` 模式具有相同的结果

Shell 中的命令：

```
>log.txt
```

会以截断写的方式打开 `log.txt`，因此可以清空文件。它与“以 `w` 模式打开文件，然后立即关闭”的结果相同。

2.3 写入与追加

2.3.1 使用 `fputs` 或 `fwrite` 写内容

以 `"a"` 模式打开文件，多次调用 `fputs`，内容会依次追加到文件末尾：

```c
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("log.txt", "a");
    if (fp == NULL)
    {
        perror("fopen");
        return 1;
    }

    for (int i = 1; i <= 10; ++i)
    {
        fputs("hello world\n", fp);
    }

    fclose(fp);
    return 0;
}
```

Shell 中的追加重定向：

```shell
cat "bbbb" >> log.txt
```

对应的效果也是保留原内容，并把新内容写到文件末尾。

2.3.2 `r+` 与 `a+` 的区别

`r+` 同时允许读和写，但要求文件已经存在，不能单独依靠 `r+` 创建文件。

`a+` 同时允许读和追加。文件不存在时会创建；输出总是追加到文件末尾，而初始读取位置在文件开头。

2.4 读取文件内容

2.4.1 使用 `fgets` 按行读取

文本文件可以先按行读取：

```c
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("log.txt", "r");
    if (fp == NULL)
    {
        perror("fopen");
        return 1;
    }

    char buffer[128];
    while (fgets(buffer, sizeof(buffer), fp) != NULL)
    {
        printf("from file: %s", buffer);
    }

    fclose(fp);
    return 0;
}
```

`fgets` 读取到缓冲区容量上限、换行符或文件末尾时返回。返回 `NULL` 可能表示到达文件末尾，也可能表示发生读取错误，必要时还要结合 `feof` 和 `ferror` 判断。

```
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ ./openfile
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ cat log.txt
hello word
hello word
hello word
hello word
hello word
hello word
hello word
hello word
hello word
hello word
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ vim openfile.c
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ make
gcc -o openfile openfile.c
ZL@VM-0-14-ubuntu:~/code/lesson13-file$ ./openfile
from file: hello word

from file: hello word

from file: hello word

from file: hello word

from file: hello word

from file: hello word

from file: hello word

from file: hello word

from file: hello word

from file: hello word

ZL@VM-0-14-ubuntu:~/code/lesson13-file$
```

2.4.2 使用 `fread` 读取任意字节

`fread` 更适合按字节块读取文件：

```c
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("log.txt", "rb");
    if (fp == NULL)
    {
        perror("fopen");
        return 1;
    }
```

```c
    char buffer[1024];
    size_t count;
    while ((count = fread(buffer, 1, sizeof(buffer), fp)) > 0)
    {
        fwrite(buffer, 1, count, stdout);
    }

    if (ferror(fp))
    {
        perror("fread");
    }

    fclose(fp);
    return 0;
}
```

这个循环就是一个简易的 `cat`：每次从文件中读出一段字节，再原样写到标准输出。

> **Tip**
>
> 不要在循环开始前直接用 `while (!feof(fp))` 作为结束条件。`feof` 只有在一次读取已经碰到文件末尾之后才会变为真，容易导致多读一轮。

```c
FILE *fopen(const char *restrict pathname, const char *restrict mode);
FILE *fdopen(int fd, const char *mode);
FILE *freopen(const char *restrict pathname, const char *restrict mode,
              FILE *restrict stream);
```

2.4.3 文件内容可以看作字节序列

从操作角度看，文件内容是一段连续字节序列，读写位置可以类比为数组下标。

因此，文件 I/O 可以理解为：

1. 从某个位置开始读一段字节；
2. 或向某个位置写一段字节；
3. 操作完成后，读写位置相应移动。

文件内容如何处理由程序决定。文本还是二进制，是程序对同一段字节序列的解释方式。

2.5 向显示器输出信息

2.5.1 C 标准库提供多种输出方式

向显示器输出信息，可以使用 `fwrite`、`printf`、`fprintf` 等接口：

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    const char *msg = "hello fwrite\n";
    fwrite(msg, 1, strlen(msg), stdout);

    printf("hello printf\n");
    fprintf(stdout, "hello fprintf\n");
    return 0;
}
```

这里的 `stdout` 是 C 标准库预先打开的标准输出流，类型同样是 `FILE *`。

2.5.2 标准输入、标准输出和标准错误

C 程序启动时通常会准备好三个标准流：

标准流	含义	常见来源或去向
`stdin`	标准输入	键盘
`stdout`	标准输出	显示器、重定向后的文件
`stderr`	标准错误	显示器、重定向后的文件

本章只把它们看作 C 标准库提供的三个 `FILE *` 文件流。它们与底层数字标识、重定向以及文件描述符之间的关系，放到下一章继续讨论。

2.6 正确关闭文件

2.6.1 `fclose` 结束一次文件流的使用

文件使用完毕后应调用 `fclose`：

```
fclose(fp);
```

关闭文件会结束这次文件流的使用并释放相关资源，同时处理 C 标准库中与该流有关的状态。对于写入操作，关闭前的内部缓冲处理尤其重要；缓冲区机制将在下一章展开。

3. 文件位置与基础读写接口回顾

3.1 读写位置的直观理解

3.1.1 文件是一维字节空间

可以把文件内容看成一段一维字符数组。所谓“当前读写位置”，就是这个一维空间中的偏移量。

每次读取或写入一部分数据后，位置通常会继续向后移动，所以下一次操作会从新的位置开始。

3.1.2 追加模式固定从末尾写入

普通写入模式可以从当前位置修改内容；追加模式则把输出位置放到文件末尾，已有内容不会被覆盖。这就是日志通常使用追加模式的原因。

3.2 C 标准库的文件位置接口

3.2.1 `ftell` 获取当前位置

`ftell` 返回当前文件流位置相对于文件开头的偏移量：

```
long pos = ftell(fp);
```

3.2.2 `fseek` 设置当前位置

`fseek` 可以相对文件开头、当前位置或文件末尾移动：

```c
fseek(fp, 0, SEEK_SET); // 移到文件开头
fseek(fp, 0, SEEK_CUR); // 保持当前位置
fseek(fp, 0, SEEK_END); // 移到文件末尾
```

3.2.3 `rewind` 回到文件开头

`rewind(fp)` 会把当前位置重新设置到文件开头，并清除与文件流有关的错误标志。

3.3 本章边界

本章已经回答的是：

1. 文件是什么；
2. 文件为什么必须由进程打开；
3. 相对路径如何借助 `cwd` 找到文件；
4. C 标准库如何打开、关闭、读写和定位文件；
5. 文件内容为什么可以看作带有位置的一维字节序列。

下一章将从“语言层的文件接口最终依赖谁操作磁盘”继续，整理 Linux 系统 I/O、文件描述符、重定向、缓冲区和“一切皆文件”的底层实现。